

Persona-based Prompting

Module 3, Chapter 2

Module 3: Hands-on with LLMs

There's one prompting technique that consistently produces better results than almost anything else in the toolkit — and it takes about five extra words. Assigning a persona to the model changes how it thinks, how it writes, and what it assumes you already know. This chapter is about how to use that deliberately.

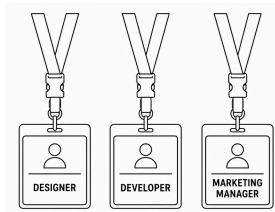
You are...

Setting the lens

"You are a patient teacher explaining this to someone with no technical background."

"You are a senior engineer doing a code review."

"You are a skeptical customer who needs convincing."



Persona prompting means telling the model who it is before you ask it anything. Not what to do — who to be. "You are a patient teacher." "You are a senior engineer." "You are a skeptical customer." That single sentence changes the vocabulary the model reaches for, the assumptions it makes about what you know, and the angle it takes on whatever you ask next. You're not changing the model. You're changing the lens.

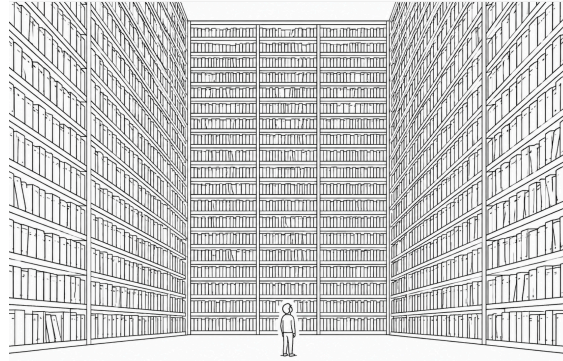
**Same Question.
Different Answers.**
Explain what an API is

Patient Teacher	"Think of an API like a waiter in a restaurant..."
Senior Engineer	"An API defines a contract between services — the interface through which..."
Business Consultant	APIs are how companies expose their capabilities to partners and developers..."

Here's the same question — "explain what an API is" — run through three personas. The patient teacher reaches for an analogy. The senior engineer goes straight to contracts and interfaces. The business consultant frames it as a strategic asset. None of these responses is wrong. They're all accurate. But only one is right for your audience at any given moment. The persona is how you tell the model which one you need.

Why it Works

The model has read everything



The reason persona prompting works so well is that the model has been trained on an enormous amount of text written by an enormous variety of people. It has read how doctors explain diagnoses to patients, how lawyers write briefs, how teachers scaffold complex ideas for beginners. When you assign a persona, you're activating that pattern set — telling the model which writing style, which vocabulary, which assumptions to pull forward. It's not pretending. It's pattern-matching at scale.

Personas for Developers

Practical prompts you can use today

"You are a senior engineer reviewing this code for readability and edge cases."

"You are a security auditor looking for vulnerabilities in this implementation."

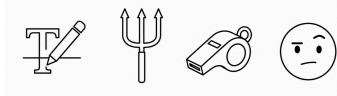
"You are a rubber duck — listen to my explanation and ask clarifying questions."

"You are a technical architect evaluating this design for scalability."

For developers, persona prompting is particularly useful because the same code or design can be evaluated through completely different lenses. A code reviewer catches readability issues. A security auditor catches vulnerabilities. A rubber duck — yes, this works — asks you questions that help you find your own bugs. And a technical architect thinks about whether the whole thing will hold up at scale. Same input, four completely different and complementary outputs.

Personas for Non-Developers

The technique isn't just for code



"You are a sharp editor who cuts ruthlessly for clarity."

"You are a devil's advocate — find every flaw in this argument."

"You are a coach helping me prepare for a difficult conversation."

"You are a skeptical customer who needs to be convinced this product is worth it."

The technique isn't just for technical work. A sharp editor will cut your writing in ways a polite model won't. A devil's advocate will find the holes in your argument before your boss does. A coach will help you think through a difficult conversation rather than just writing a script for you. And a skeptical customer will tell you whether your product pitch actually lands. These personas work because they give the model permission to be direct in ways it otherwise tends to avoid.

Personas Stack

Combine with what you already know

Persona + few-shot: "You are a sharp editor. Here are two examples of how I like my writing trimmed: [examples]. Now edit this paragraph."

Persona + chain-of-thought: "You are a security auditor. Think through this code step by step, flagging each potential vulnerability before summarizing."

Persona prompting stacks cleanly with everything we covered in Module 2. Persona plus few-shot gives the model both a role and examples of how that role should perform — extremely consistent output. Persona plus chain-of-thought asks the model to work through a problem in character, step by step, which catches errors that a direct answer would skip. Once you're comfortable with persona prompting on its own, start combining it. That's where the real control comes from.

What Breaks it

Mistakes worth avoiding



Too vague	Be helpful and smart	->	No change in behavior
Too fictional	You are a wizard from the future	->	Novelty without utility
Conflicting instructions	You are a concise editor. Write me a 2,000 word analysis	->	The model picks one

Three things that break persona prompting. Too vague — "be helpful and smart" gives the model nothing to work with because it's already trying to be helpful and smart. Too fictional — "you are a wizard from the future" produces novelty but not utility; the persona needs to map to a real-world expertise. And conflicting instructions — asking a concise editor to write a comprehensive 2000-word analysis forces the model to pick one and ignore the other. Make the persona specific, real, and consistent with the rest of your prompt.

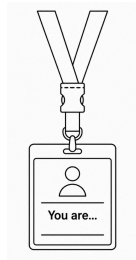
Try it now

Run this in your preferred LLM

Without persona	"Explain the pros and cons of remote work."
With persona	"You are an HR director who has managed both remote and in-office teams for ten years. Explain the pros and cons of remote work for a company considering a permanent hybrid policy. Be direct and practical. Under 200 words."

Here's one to try right now. Run the weak version first — "explain the pros and cons of remote work" — and note what comes back. Then run the persona version. Same topic, but now the model is an experienced HR director speaking to a specific decision-maker about a specific policy question. The difference in focus, tone, and usefulness is immediate. Try it with your own topic — pick any expertise you'd find useful and assign it before your next prompt.

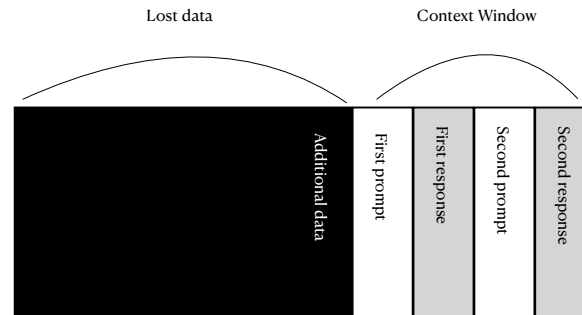
Key Takeaways



- "You are..." sets the lens
- Activates vocabulary, assumptions, and tone
- Works for technical and non-technical tasks
- Stacks with few-shot and chain-of-thought
- Keep personas specific, real, and consistent

Two words — "you are" — and everything that follows changes. The vocabulary shifts. The assumptions shift. The depth and angle shift. It works for code review and copyediting, for technical architecture and difficult conversations. It stacks with every other technique we've covered. And it takes five seconds to add to any prompt you're already writing. In the next chapter we'll look at the places where even good prompting runs into walls — how models handle memory, where bias shows up, and what to do when you need to start fresh.

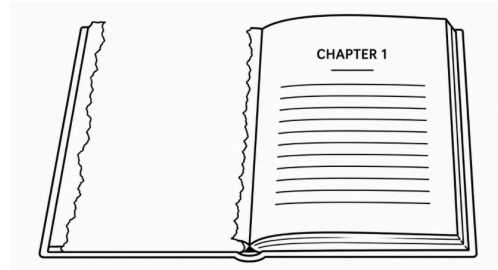
Context Length Explained



The context window is the model's working memory — everything it can see at once. Your prompt, the conversation history, any documents you've passed in — all of it counts against that limit. Older models topped out around 4,000 tokens. Many current models handle hundreds of thousands. But the rule holds: if it's outside the window, the model has no idea it ever existed.

Lost Context

Repetition or forgetting



When a conversation runs long enough that earlier content falls out of the window, the model loses access to it entirely. That's why you'll sometimes see a model contradict something it said twenty messages ago — it literally can't see that far back anymore. And even when you're well within the window, cramming in too much information at once can dilute the model's focus and produce vaguer, less useful responses.

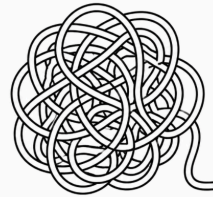
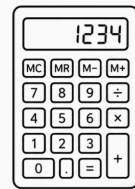
Reasoning and its Limits

Pattern

Logical

Recognition

Deduction



The good news is reasoning can be scaffolded. Chain-of-thought prompting — breaking a problem into explicit steps — meaningfully improves accuracy. Giving the model access to tools like calculators removes the need to "do math" at all. And RAG, which we'll spend a lot of time on later in this course, grounds responses in verified external data. The model reasons better when you structure the problem for it.

Improving Reasoning Performance

All together now



Three limitations worth keeping in your head whenever you work with LLMs. Hallucinations happen when the model lacks grounding. Context length defines what the model can actually see and remember. And reasoning is simulated — fluent and convincing, but not foolproof. Understanding these isn't about being cynical about the technology. It's about using it well. And building systems that account for where it fails is exactly what the rest of this course is about.